

LECTURE 10

Relational Calculus

Describing what an answer looks like, not how to compute it

CSC270 · Database Systems

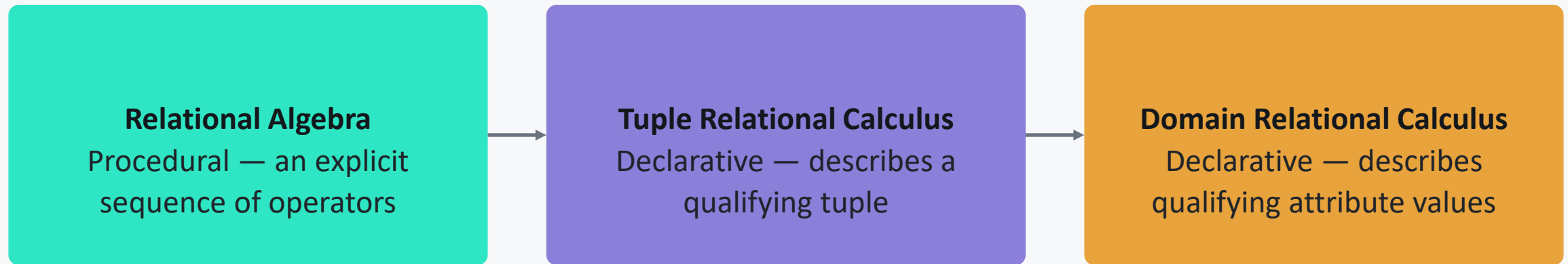
COMSATS University Islamabad, Lahore Campus

What This Lecture Covers

- 1 Relational calculus as a declarative query language, vs. algebra's procedural style
- 2 Codd's theorem: algebra, tuple calculus, and domain calculus are equally expressive
- 3 Tuple Relational Calculus (TRC): tuple variables, $\{ T \mid F(T) \}$
- 4 Domain Relational Calculus (DRC): one domain variable per attribute
- 5 Quantifiers: existential ($\exists$) and universal ($\forall$), written $\neg\exists\neg$
- 6 Safety of expressions: why some valid formulas are meaningless

Procedural vs. Declarative

- Relational algebra is procedural: π name (σ salary > 20000 (Staff)) names an explicit sequence of steps.
- Relational calculus is declarative: you write a description of what a qualifying tuple looks like, and leave every “how” decision to the DBMS's optimizer.



Two languages, one expressive power — Codd's theorem

Codd's Theorem

NOTE

E. F. Codd proved that relational algebra and relational calculus (restricted to safe expressions) have exactly the same expressive power: any query expressible in one is expressible in the other, and vice versa.

- “Write this query in algebra” and “write this query in calculus” are two views of the same underlying question — never two different problems.
- SQL, met properly in Unit 6, is itself calculus-flavored: WHERE salary > 20000 is a description, not a step-by-step procedure.

Both calculus dialects come from predicate calculus (first-order logic): a query is a formula that is either true or false for a given tuple, and the result is everything for which the formula evaluates true.

Tuple Relational Calculus and Tuple Variables

```
{ T | F(T) }
```

- Read as: “the set of all tuples T such that formula F(T) is true.”
- T is a tuple variable — it ranges over an entire relation, one whole row at a time.
- Individual attribute values are accessed with dot notation: T.salary, T.name — exactly like a struct field.

Staff (staffNo, name, position, salary, branchNo)

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SL41	Julie Lee	Manager	22000	B005

EXAMPLE A

Selection and Projection

“Find the names and salaries of staff earning more than 20000”

```
TRC:      { S.name, S.salary | Staff(S)  $\wedge$  S.salary > 20000 }
```

```
Algebra:  $\pi$  name, salary ( $\sigma$  salary > 20000 (Staff))
```

Result — both expressions

name	salary
John White	30000
Julie Lee	22000

NOTE

Staff(S) restricts S to range only over tuples of the Staff relation — every valid TRC formula must include this kind of “range” condition, or the query is meaningless (see Safety, later).

Tracing Example A, Row by Row

Check each Staff row against salary > 20000, one row at a time, the way a DBMS scans a relation

```
TRC:      { S.name, S.salary | Staff(S) ∧ S.salary > 20000 }  
Algebra:  $\pi$  name, salary ( $\sigma$  salary > 20000 (Staff))
```

Row-by-row check

Staff row	Salary > 20000?
SL21 John White (30000)	yes
SG37 Ann Beech (12000)	no
SG14 David Ford (18000)	no
SA9 Mary Howe (9000)	no
SL41 Julie Lee (22000)	yes

NOTE

Only the rows marked yes pass the selection. The projection then keeps name and salary, so the final result is exactly John White and Julie Lee.

EXAMPLE B

Selecting Whole Tuples

“Find all staff who work at branch B003” — want entire rows back, not a subset of columns

```
TRC:      { S | Staff(S)  $\wedge$  S.branchNo = 'B003' }
```

```
Algebra:  $\sigma$  branchNo = 'B003' (Staff)
```

Result — both expressions above

staffNo	name	position	salary	branchNo
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003

Domain Relational Calculus and Domain Variables

$$\{ x_1, x_2, \dots, x_n \mid F(x_1, x_2, \dots, x_n) \}$$

- DRC uses one variable per attribute value — a domain variable — instead of one variable per tuple.
- Each x_i ranges over the domain of one attribute.
- Any domain variable appearing in the result must be free; every other variable needed to state relation membership must be existentially quantified with $\exists$.

TRC vs. DRC — Same Query, Two Notations

TRC — one variable, dot notation

```
{ S.name, S.salary |  
  Staff(S)  $\wedge$  S.salary > 20000 }
```

DRC — one variable per attribute

```
{ name, salary |  
   $\exists$ sNo, pos, bNo  
  (Staff(sNo, name, pos,  
    salary, bNo)  
   $\wedge$  salary > 20000) }
```

Both formulas describe the identical set of (name, salary) pairs, and both equal π name, salary (σ salary>20000 (Staff)) — Codd's theorem made concrete: three notations, one answer.

Existential Quantification Across Relations

- Calculus becomes more expressive than a single σ/π pair once a query must reach into a second relation — exactly what a join does in algebra.

Client (**clientNo**, **name**, **telNo**, **prefType**, **maxRent**)

clientNo	name	prefType	maxRent
CR76	John Kay	Flat	30000
CR56	Aline Stewart	Flat	27000
CR74	Mike Ritchie	House	45000

Viewing: The Relation That Links Them

Viewing (clientNo, propertyNo, viewDate, comment)

clientNo	propertyNo	viewDate
CR76	PA14	2026-05-10
CR56	PG4	2026-05-12
CR56	PG36	2026-05-13
CR74	PG21	2026-05-15
CR74	PG4	2026-05-16
CR74	PG36	2026-05-17

PG4, PG36, PG21 are all in Lahore (branch B003); PA14 is in Islamabad, PL94 is in Karachi.

Clients Who Viewed a Lahore Property

TRC:

```
{ C.name | Client(C) ∧ (∃V) (Viewing(V) ∧ V.clientNo = C.clientNo ∧  
    (∃P) (PropertyForRent(P) ∧ P.propertyNo = V.propertyNo ∧ P.city =  
    'Lahore')) }
```

```
Algebra:  $\pi$  name ( Client  $\bowtie$  ( Viewing  $\bowtie$  ( $\sigma$  city='Lahore' (PropertyForRent))  
    ) )
```

- CR56 (Aline Stewart) viewed PG4 and PG36 — both Lahore, so she qualifies.
- CR74 (Mike Ritchie) viewed PG21, PG4, PG36 — all Lahore, so he qualifies too.
- CR76 (John Kay) viewed only PA14, in Islamabad — he does not qualify.

Result

name
Aline Stewart
Mike Ritchie

Universal Quantification

- $\exists$ asks “does at least one match exist?” Its partner $\forall$ asks “does every one match?”
- Relational calculus has no separate $\forall$ symbol in its common presentation — it is expressed with the identity:

$$\forall x (P(x)) \equiv \neg \exists x (\neg P(x))$$

“Every x satisfies P ” means exactly “there is no x that fails P .” This is the natural way to state questions like “clients who have viewed every property a branch manages.”

EXAMPLE D

Clients Who Viewed Every B003 Property

Branch B003's properties are {PG4, PG36, PG21}.

```
{ C.name | Client(C) ∧  
    ¬(∃P) (PropertyForRent(P) ∧ P.branchNo = 'B003' ∧  
        ¬(∃V) (Viewing(V) ∧ V.clientNo = C.clientNo ∧ V.propertyNo =  
P.propertyNo)) }
```

- CR74 (Mike Ritchie) viewed all three B003 properties — no unviewed property exists for him; he qualifies.
- CR56 (Aline Stewart) never viewed PG21 — that missing viewing is the counter-example; she fails.
- CR76 (John Kay) viewed none of the three — fails immediately.

Result

name
Mike Ritchie

NOTE

Algebra equivalent: division (÷). It returns the same single-row answer, reached mechanically.

Safety of Expressions

```
{ S | ¬Staff(S) }
```

- Read literally: “every tuple that is not in Staff” — an infinite set of every possible combination of values.
- A formula whose result could be infinite, or whose truth depends on values outside any named relation, is unsafe.
- A formula is safe only when its result is guaranteed finite and drawn entirely from the domains of the relations it mentions.

WARNING

Codd's theorem only covers safe expressions. Every algebra expression translates to a safe calculus formula, but a careless formula (like the negation above) may have no algebra equivalent at all.

PRACTICE

Your Turn: Write the Query

Find all staff who work at branch B005. Write the TRC and algebra yourself before the next slide.

```
TRC:      { ... }  
Algebra: ...
```

Pause and think

staffNo	name	position	salary	branchNo
?	?	?	?	?
?	?	?	?	?

Answer: Staff at Branch B005

Keep every Staff row whose branchNo is B005 — both columns and rows come back, so the whole tuple is kept.

```
TRC:      { S | Staff(S) ∧ S.branchNo = 'B005' }
```

```
Algebra:  $\sigma_{\text{branchNo} = \text{'B005'}}(\text{Staff})$ 
```

Result

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SL41	Julie Lee	Manager	22000	B005

Every Query, Three Notations

Query	TRC	Algebra
Staff > 20000, name & salary	$\{S.name, S.salary \mid Staff(S) \wedge S.salary > 20000\}$	$\pi name, salary (\sigma salary > 20000 (Staff))$
All staff at branch B003	$\{S \mid Staff(S) \wedge S.branchNo = 'B003'\}$	$\sigma branchNo = 'B003' (Staff)$
Clients who viewed a Lahore property	nested $\exists$ across Client, Viewing, PropertyForRent	nested joins, filtered first with σ
Clients who viewed every B003 property	uses $\neg\exists\neg$ (universal quantification)	division ($\div$)

KEY TAKEAWAYS

Relational Calculus

- 1 Calculus is declarative (“what the answer looks like”) vs. algebra's procedural style — Codd's theorem guarantees equal power over safe expressions.
- 2 TRC queries have the form $\{T \mid F(T)\}$, where T is a tuple variable accessed with dot notation.
- 3 DRC uses one domain variable per attribute value, existentially quantifying every attribute not in the result.
- 4 $\exists$ asks “does at least one match exist”; $\forall$, written $\neg\exists\neg$, asks “does every one match” — and corresponds to algebra's division.
- 5 A formula is safe only when finite and every variable is tied to a named relation — unsafe formulas have no algebra equivalent.